Web Application Development

- React Routing, State Management

Ammar Ahmad Khan

Vite

Vite (pronounced "veet") is a modern **build tool and development server** for web applications.

It was created by Evan You (the creator of Vue.js), but it's now widely used with **React**, **Svelte**, **Vue**, and even **Vanilla JS**.

Can REACT work Alone?

No (in real projects).

React is just a **library**, not a complete system.

It helps you:

- Create components
- Manage UI
- Handle state

But it **does NOT handle**:

- Combining multiple files
- Converting modern JavaScript (ES6, JSX) into browser-compatible code
- Optimizing code for production
- Loading CSS, images, assets

That's where **build tools** come in.

What is Build?

A **build** means: Converting your development code into a version that browsers can understand and run efficiently.

Example:

You write:

```
const App = () => <h1>Hello</h1>;
```

Browser sees:
"I don't understand JSX"

Build process converts it into:

```
const App = function() {
  return React.createElement("h1", null, "Hello");
};
```

Now browser understands it.

What is Bundler?

A **bundler** takes many files and combines them into fewer files.

Imagine your project:

App.jsx
Header.jsx
Footer.jsx
styles.css
utils.js

A bundler (like Webpack or Rollup):

Combines them into:

bundle.js
style.css

Why Bundling is needed?

Without bundling:

- Browser would make **many requests** (slow)
- Code dependencies become messy
- Hard to optimize performance

With bundling:

- Faster loading
- Cleaner dependency handling
- Optimized output

What Vite does for you:

1. Handles Build

- Converts JSX → browser JavaScript
- Optimizes code

2. Handles Bundling

- Combines files for production

3. Runs Dev Server

- Shows your app in browser

4. Fast Updates

- Instant reload when you change code

Why Use Vite with React?

React projects need a bundler and dev server to:

- Compile modern JavaScript (like JSX).
- Serve files during development.
- Bundle assets for production.

Traditionally, this was done using **Webpack** (like in create-react-app), but Vite is much **faster and more efficient.**

Bundler?

A **bundler** is a tool that:

Takes all your source files (JavaScript, CSS, images, etc.)
Combines them into a few optimized files (often just one or two)
So that the browser can load your app **efficiently**.

Think of it like:

Turning 50 small scattered ingredients (JS files, modules) into 1 ready-made meal (bundle.js).

Note: WebPack and Rollup is a javascript bundler and build tool.

Advantages of Using Vite with React

Fast Startup: Instantly starts the dev server: no waiting like in **Webpack**.

Hot Module Replacement (HMR): Edits appear instantly without reloading the entire app.

Uses ES Modules in Dev: Faster because **Vite** serves unbundled native modules in development.

Lightning Fast Builds: Uses **Rollup** under the hood for optimized production builds.

Simpler Config: Less boilerplate, easier to customize than **create-react-app.**

Framework Agnostic: Works with React, Vue, Svelte, etc.

Why Vite with React is Best?

Vite is a **next-generation toolchain** that makes React development **faster, simpler, and more enjoyable**.

If you're starting a new project today, using **Vite + React** is generally a better choice than create-react-app.

Steps to configure Vite with React

1. Install Node.js

Make sure you have **Node.js (v16 or later)** installed.

- Download from: https://nodejs.org/

- After installing, check:

```
node -v
npm -v
```

Steps to configure Vite with React

2. Create a Vite + React Project

Open a terminal (Mac: Terminal app, Windows: CMD or PowerShell), and run:

```
npm create vite@latest
```

It will prompt:

- **Project name?** → my-app (or any name you like)
- **Select a framework:** → React
- **Select a variant:** → JavaScript or TypeScript (choose JavaScript for now if unsure)

Then go into your project folder:

```
cd my-app
```

Steps to configure Vite with React

3. Install dependencies

Now install the packages:

```
npm install
```

4. Start development server

```
npm run dev
```

- It will output a local URL like:

  Local: http://localhost:5173/

Open this in **Google Chrome** or any browser.

React Router

- Create React App **doesn't** include page routing.
- React Router is the most popular **solution**.

Add React Router

To add React Router in your application, run this in the terminal from the root directory of the application:

npm i -D react-router-dom

Note: I uses React Router v6.

If you are upgrading from v5, you will need to use the @latest flag:

npm i -D react-router-dom@latest

Need to install React Router if Vite is already installed?

When you're using **Vite + React**, installing react-router-dom is **not automatic:** you must install it separately when you need routing.

Add React Router when needed:

If you plan to use routing (for pages like Home, Blog, Contact), install:

npm install react-router-dom

Why it's separate?

Because:

- Not all projects need routing.
- Keeping it separate keeps your app lightweight unless you **actually need React Router**.

Folder Structure

To create an application with multiple page routes, let's first start with the file structure.

- **Within the src folder, we'll create a folder named pages with several files:**

  src\pages\:

  - Layout.js
  - Home.js
  - Blogs.js
  - Contact.js
  - NoPage.js

- Each file will contain a very basic React component.

Basic Usage

Now we will use our Router in our **main.jsx** file.

Example

Use **React Router** to route to pages based on URL:

main.jsx:

```
import ReactDOM from "react-dom/client";
import { BrowserRouter, Routes, Route } from "react-router-dom";
import Layout from "./pages/Layout";
import Home from "./pages/Home";
import Blogs from "./pages/Blogs";
import Contact from "./pages/Contact";
import NoPage from "./pages/NoPage";

export default function App() {
  return (
    <BrowserRouter>
      <Routes>
        <Route path="/" element={<Layout />}>
          <Route index element={<Home />} />
          <Route path="blogs" element={<Blogs />} />
          <Route path="contact" element={<Contact />} />
          <Route path="*" element={<NoPage />} />
        </Route>
      </Routes>
    </BrowserRouter>
  );
}

const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(<App />);
```

Example Explained (main.jsx)

We wrap our content first with <BrowserRouter>.

Then we define our <Routes>. An application can have multiple <Routes>. Our basic example only uses one.

<Routes> can be nested. The first <Route> has a path of / and renders the Layout component.

The nested <Routes> inherit and add to the parent route. So the blogs path is combined with the parent and becomes/blogs.

The Home component route does not have a path but has an index attribute. That specifies this route as the default route for the parent route, which is /.

Setting the path to * will act as a catch-all for any undefined URLs. This is great for a 404 error page.

Pages / Components

The Layout component has <Outlet> and <Link> elements.

The <Outlet> renders the current route selected.

<Link> is used to set the URL and keep track of browsing history.

Anytime we link to an internal path, we will use <Link> instead of <a href="">.

The "layout route" is a shared component that inserts common content on all pages, such as a navigation menu.

Layout.jsx

```jsx
import { Outlet, Link } from "react-router-dom";

const Layout = () => {
  return (
    <>
      <nav>
        <ul>
          <li>
            <Link to="/">Home</Link>
          </li>
          <li>
            <Link to="/blogs">Blogs</Link>
          </li>
          <li>
            <Link to="/contact">Contact</Link>
          </li>
        </ul>
      </nav>

      <Outlet />
    </>
  )
};

export default Layout;
```

Home.jsx

```jsx
const Home = () => {
  return <h1>Home</h1>;
};

export default Home;
```

Blogs.jsx

```
const Blogs = () => {
  return <h1>Blog Articles</h1>;
};

export default Blogs;
```

Contact.jsx

```jsx
const Contact = () => {
  return <h1>Contact Me</h1>;
};

export default Contact;
```

NoPage.jsx

```
const NoPage = () => {
  return <h1>404</h1>;
};

export default NoPage;
```

Tasks 15-16

Open Lecture 15-16 Tasks doc file uploaded on QOBE

Integrating React with Bootstrap

Go to React Bootstrap website: https://react-bootstrap.netlify.app/docs/getting-started/introduction

First install **bootstrap** via **NPM**:

Installation

The best way to consume React-Bootstrap is via the npm package which you can install with `npm` (or `yarn` if you prefer).

If you plan on customizing the Bootstrap Sass files, or don't want to use a CDN for the stylesheet, it may be helpful to install vanilla Bootstrap as well.

```
npm install react-bootstrap bootstrap
```

Integrating React with Bootstrap

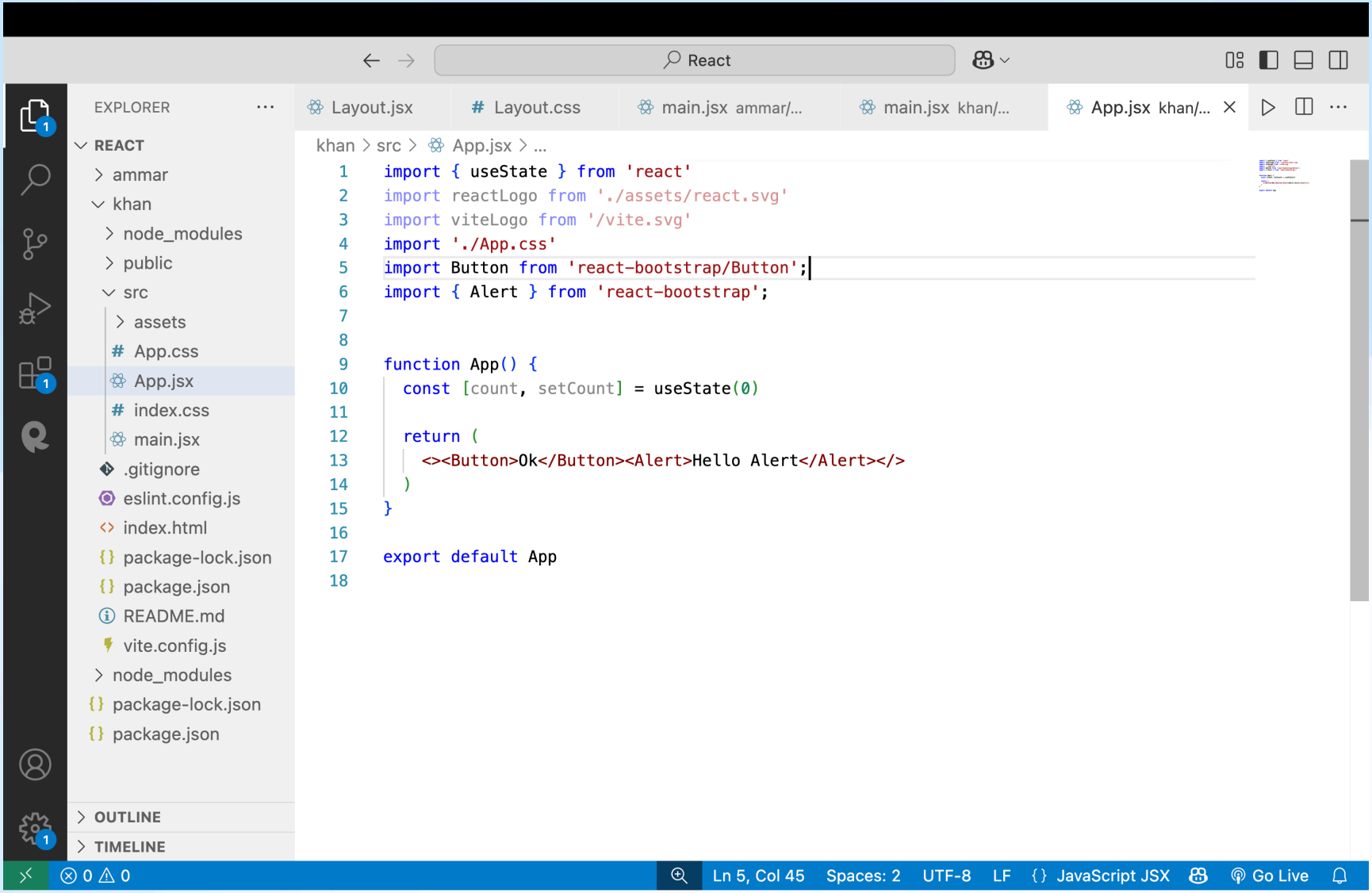
Import css in app.js or index.js:

CSS

```
{
  /* The following line can be included in your src/index.js or App.js file */
}
import 'bootstrap/dist/css/bootstrap.min.css';
```

You can add any bootstrap component now by following the instructions on the web.
Link: https://react-bootstrap.netlify.app/docs/getting-started/introduction

Adding Button and apply style



Context API

What is an API?

- Stands for Application Programming Interface
- A way to exchange data between applications

What is Context?

- Related to *content* or *environment*
- In React: shared data accessible across components

So, Context API = A way to share content/data between components without props.

Why Do We Need Context API?

Problem with Props:

- Props can only pass data **from parent to direct child**.
- Deeply nested components need "**prop drilling**" (manually passing props layer by layer).

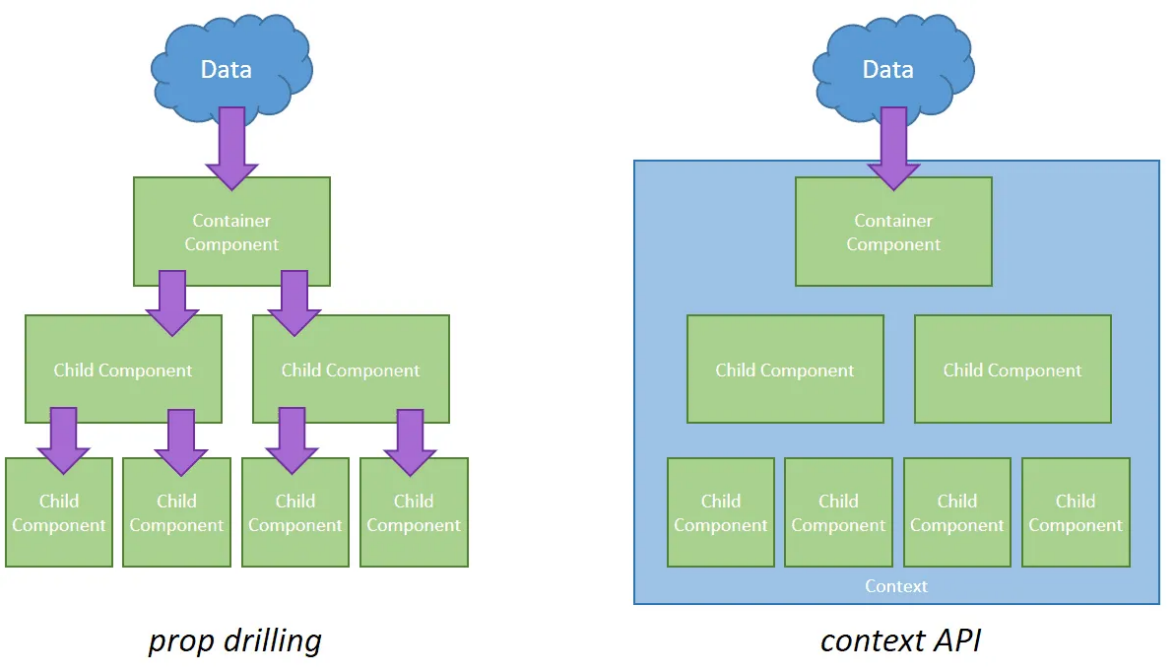
Limitation: Not scalable when components are not directly related.

Prop Drilling vs Context API

With Props:
Home → About → ContactUs

With Context API:
Home → (Context Provider) → ContactUs

- No need for direct parent-child relationship



Example 1: Prop Drilling (Bad Practice When Overused)

Scenario:

We have a component hierarchy like this:

<App>
  └── <Dashboard>
    └── <Sidebar>
      └── <UserProfile>

We want to show the user's name in `<UserProfile>`, but the user data comes from `<App>`.

Prop Drilling Code (Messy)

```
// App.js
function App() {
  const user = { name: "Alice" };
  return <Dashboard user={user} />;
}
// Dashboard.js
function Dashboard({ user }) {
  return <Sidebar user={user} />;
}
// Sidebar.js
function Sidebar({ user }) {
  return <UserProfile user={user} />;
}
// UserProfile.js
function UserProfile({ user }) {
  return <h2>Hello, {user.name}!</h2>;
}
```

Provider and Consumer

1. Provider – *"The Data Giver"*

- It wraps your component tree and **provides the context value** to all its children.
- You use this to **share** data like theme, user, language, etc.

Example:

```
<ThemeContext.Provider value={"dark"}>
  <Header />
</ThemeContext.Provider>
```

value={"dark"} is shared with everything inside it (<Header /> and its children).

Provider and Consumer

2. Consumer – *"The Data Receiver"*

- It's used to **read** the value from the nearest Provider above in the tree.
- You use either:
  - `useContext()` (modern way)
  - `<ThemeContext.Consumer>` (older way, used before hooks)

Old Way (Before React Hooks)

```
<ThemeContext.Consumer>
  {value => <p>Theme is {value}</p>}
</ThemeContext.Consumer>
```

This syntax is bulky and hard to nest.

Modern Way (Using `useContext()`)

```
const theme = useContext(ThemeContext);
```

Why do need Provider and Consumer?

Provider:

Shares data with all child components that need it (like setting up Wi-Fi in your app)

Consumer:

Receives and uses that data inside any child component (like connecting your phone to the Wi-Fi)

Example 2: Context API (Clean Solution)

Scenario:

Same layout as above — but now we use **Context** to avoid passing `user` manually.

Step-by-Step

1. Create Context

```
// UserContext.js
import { createContext } from 'react';
export const UserContext = createContext();
```

2. Provide Context

```
// App.js
import { UserContext } from './UserContext';
import Dashboard from './Dashboard';

function App() {
  const user = { name: "Alice" };

  return (
    <UserContext.Provider value={user}>
      <Dashboard />
    </UserContext.Provider>
  );
}
```

3. Consume Context

```
// UserProfile.js
import { useContext } from 'react';
import { UserContext } from './UserContext';

function UserProfile() {
  const user = useContext(UserContext);
  return <h2>Hello, {user.name}!</h2>;
}
```

No need to pass `user` through `Dashboard` or `Sidebar` anymore!